



Ends in:
1d 07h 01m 50s



EN

TUTORIALS ▾

category ▾

[Home](#) > [Tutorials](#) > [Artificial Intelligence](#)

A Guide to Andrej Karpathy's AutoResearch: Automating ML with AI Agents

Learn how Karpathy's AutoResearch runs 100+ ML experiments overnight on a single GPU. Covers the three-file architecture, ratchet loop, results, and limitations.

 Contents

Mar 23, 2026 · 11 min read

EXPLORE WITH AI

**Bex Tuychiev**

Bex is a Top 10 AI writer on Medium and a Kaggle Master with over 15k followers. He loves writing detailed guides, tutorials, and notebooks on complex data science and machine learning topics.

TOPICS

[Artificial Intelligence](#)[AI Agents](#)

Andrej Karpathy's AutoResearch is an [open-source tool](#) that runs ML experiments in a loop, keeping only the changes that beat the current best result. You describe research directions in a markdown file, point an AI coding agent at the repo, and walk away. By

morning, you have a git history of validated improvements and a log of everything the agent tried.

Released on March 7, 2026, the project picked up 21,000+ GitHub stars and 8.6 million views on Karpathy's announcement within days. This article covers how the three-file architecture works, what the ratchet loop does during those overnight runs, what results AutoResearch has produced so far, and where the approach hits its limits.

What is AutoResearch?

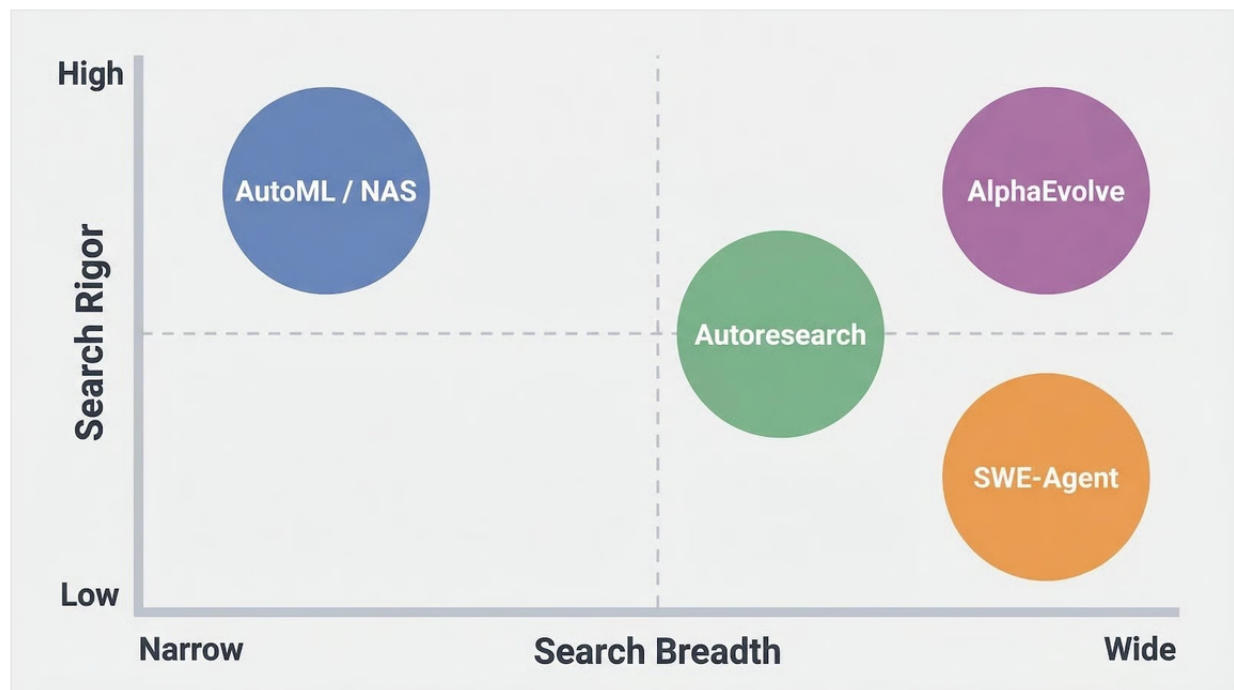
AutoResearch is an open-source Python tool that lets an AI agent run ML experiments on a single GPU without human intervention. It loops through propose-train-evaluate cycles, keeping only changes that improve validation loss. The project ships under an MIT license.

This is not [hyperparameter tuning](#). Tools like Optuna or Ray Tune search a predefined parameter space. AutoResearch gives the agent freedom to modify arbitrary code. The search space is whatever the LLM can think of, and that makes it a different category of tool from anything currently available.

[AutoML and NAS \(Neural Architecture Search\) frameworks](#) search over architectures or hyperparameters using structured algorithms, precise but constrained to their defined search space.

AlphaEvolve (Google DeepMind) goes further with an evolutionary approach and Gemini models for algorithm discovery, but it's closed-source and not accessible to most teams. General-purpose [coding agents](#) like SWE-Agent, OpenHands, and Aider can write arbitrary code but aren't built for the experiment-evaluate-keep/revert cycle that ML research actually requires.

AutoResearch bets on the LLM's general knowledge to propose good experiments, rather than constraining the search space for mathematical guarantees.



From vibe coding to research advisor

Karpathy frames this as a natural progression in how engineers work with AI. In February 2026, he coined the term "agentic engineering": *"You are not writing the code directly 99% of the time. You are orchestrating agents who do and acting as oversight."*

AutoResearch takes the next step. The human doesn't even orchestrate. They describe what good research looks like in a markdown file and walk away.

The progression goes: vibe coding (human prompts, AI writes code, human reviews) to agentic engineering (human orchestrates agents in real time) to fully independent research (human sets direction, agent runs on its own).

Each step reduces the human's role from writer to director to, in Karpathy's framing, research advisor.



In a follow-up post, he described the next step: *"The goal is not to emulate a single PhD student, it's to emulate a research community of them"*, referencing a SETI@home-style distributed agent collaboration.

How the system implements that vision comes down to three files.

AutoResearch's Three-File Architecture

AutoResearch's design comes down to a contract between three files, each with strict rules about who can touch it.

prepare.py

`prepare.py` handles data preparation and evaluation. It builds a BPE (Byte Pair Encoding) tokenizer with an 8,192-token vocabulary, processes the training corpus, and defines the validation metric: `val_bpb` (validation bits-per-byte).

This file is immutable: neither the human nor the agent modifies it, which guarantees that every experiment is measured against the same yardstick.

train.py

`train.py` is the agent's sandbox: 630 lines containing the GPT model architecture, the Muon+AdamW optimizer, and the full training loop. The agent can rewrite anything here

(swap activation functions, restructure attention heads, change learning rate schedules, modify weight initialization) as long as the modified code still trains and produces a `val_bpb` score.

program.md

`program.md` is written in plain markdown and is the only file the human author touches. It tells the agent what research directions to pursue, what to avoid, and how to approach experiments.



What `program.md` controls

The file is more specific than most people expect. It hardcodes baseline metrics so the agent knows what to beat (`val_bpb` : 0.997900, peak VRAM (video memory): 45 GB).

It specifies exact commands for running experiments and extracting results. It tells the agent how to handle failures: fix typos and re-run, skip ideas that are broken at the root, kill anything that runs past 10 minutes.

And it includes the directive that makes the whole system work: "NEVER STOP. Once the experiment loop has begun, do NOT pause to ask the human if you should continue."

There is also a design constraint that shapes every experiment: "All else being equal, simpler is better. A small improvement that adds ugly complexity is not worth it." This steers the agent away from overengineered solutions and toward clean changes a

human reviewer would approve.

The division of labor is clean. The human sets research direction through `program.md`, the agent executes by modifying `train.py`, and `prepare.py` acts as the neutral judge that neither side can touch. With that contract in place, the agent runs the experiment loop continuously without stopping.

AutoResearch Ratchet Loop

The core of AutoResearch is an experiment cycle that runs without human input. Here's how a single iteration works, following the 9-step loop defined in `program.md`:

1. The agent reads `program.md` to understand current research priorities and constraints.
2. It examines the current `train.py` and recent results in `results.tsv`.
3. It proposes a hypothesis: an architecture change, optimizer adjustment, or training modification.
4. It modifies `train.py` to implement the proposed change.
5. It commits the change to a git branch.
6. It runs training for exactly 5 minutes (fixed wall-clock budget).
7. If the training crashes, it logs the failure, reverts the commit, and tries again.
8. It evaluates the result using `val_bpb` and records the outcome in `results.tsv`.
9. If `val_bpb` improved: the commit stays. If not: `git reset HEAD~1` reverts to the previous version.

Then it starts again from step 1.



Every experiment gets the same 5-minute wall-clock budget, making results directly comparable. A change that trains faster and a change that converges lower are evaluated on equal footing.

At 5 minutes per experiment, the system runs about 12 experiments per hour.

The "ratchet" name comes from the git history. Each successful experiment adds a commit; each failure gets reverted. The codebase can only move forward, never backward, accumulating validated improvements one at a time.

Git as research memory

This resembles evolutionary algorithms in structure, but AutoResearch keeps a single lineage rather than a population. Instead of crossover and mutation across candidates, the LLM acts as both the mutation operator (proposing changes) and the selection pressure (choosing what to try based on past results).

It reads its own git history and `results.tsv` to build on whatever showed promise.

The `results.tsv` file tracks every experiment: commit hash, `val_bpb` score, GPU memory usage, pass/fail status, and a description of what the agent tried.

You get a clear audit trail to review in the morning, and the agent uses the same log to calibrate what to try next. Early experiments tend to be broad (testing different optimizer configs), while later ones narrow in on whichever direction the data validated.

Getting Started With AutoResearch

Running AutoResearch requires a machine with an NVIDIA GPU (the default config targets modern GPUs with 20+ GB VRAM), Python 3.10+, uv, and a coding agent (Claude Code, Cursor, or similar).

Clone the repo, install dependencies, and prepare the dataset:

```
git clone https://github.com/karpathy/autoresearch.git
cd autoresearch
uv sync
uv run prepare.py
```



 Explain code

POWERED BY  datalab

There is no orchestration script. No `run.py`, no pipeline, no framework.

The README says to "simply spin up your Claude/Codex or whatever you want in this repo."

You open a coding agent in the project directory, prompt it to read `program.md`, and the agent runs the experiment loop on its own.

The LLM is the automation layer. Watch progress by tailing `results.tsv` or checking the git log for new commits.

The default configuration trains a GPT model on the FineWeb-Edu dataset, which needs a decent GPU and several hours to show results. For a first test on smaller hardware, Karpathy recommends switching to the TinyStories dataset and scaling down the model by reducing vocabulary size to 256 and depth to 4.

These changes bring VRAM requirements down and shorten each experiment enough to see the ratchet in action within a couple of hours.

Configuring your first run

Read the default `program.md` before you kick off an overnight run. It defines the research agenda the agent follows, and editing it is how you steer experiments.

If you want the agent to focus on attention mechanisms, say so in `program.md`.

If you want it to avoid touching the optimizer, add that constraint. The agent will not deviate from what's written there.

For a first overnight run, expect 80-100 experiments with maybe 15-20 improvements kept. A few practical things to know:

- API costs scale with experiment count. A single overnight run is manageable for individual researchers, but multi-day runs need budgeting.
- Some experiments will crash. The loop recovers automatically, so they won't interrupt an overnight run.
- Check `results.tsv` in the morning rather than watching the terminal. The staircase pattern of improving `val_bpb` scores tells the story better than individual logs.

AutoResearch Results and the Creativity Ceiling

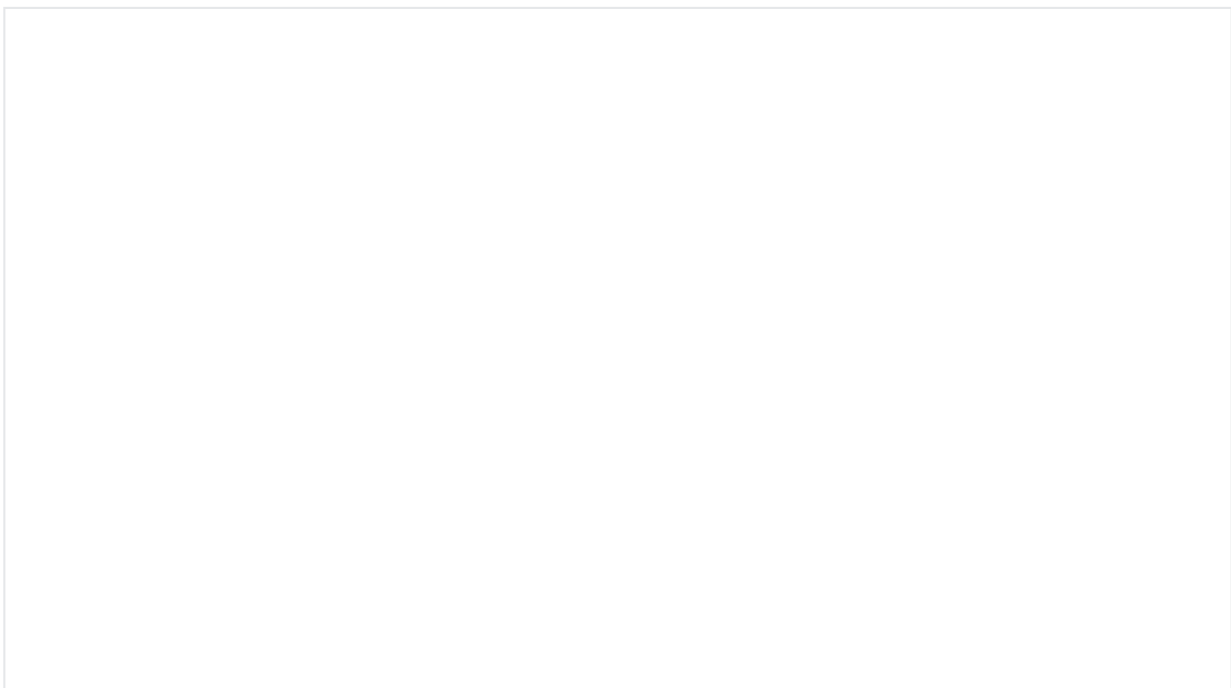
AutoResearch has been tested across multiple runs, from Karpathy's own experiments to community reproductions and production adoption.

Run	Experiments	Improvements Kept	Result
Initial overnight (single GPU)	83	15	val_bpb: 1.000 → 0.975
Extended 2-day run (depth-12)	~700	~20	All additive; transferred to depth-24 models
Production impact	-	-	Time-to-GPT-2 benchmark: 2.02h → 1.80h (11% faster)
Community session	126	-	val_bpb: 0.9979 → 0.9697

The agent found things a methodical human would eventually find. QKnorm was missing a scalar multiplier for attention sharpening, Value Embeddings benefit from regularization, and there were gains in banded attention tuning, AdamW beta parameters, and weight decay scheduling.

These are structural code changes, not random hyperparameter sweeps, and testing each one manually would have taken days.

Shopify CEO Tobi Lutke adapted AutoResearch for an internal query-expansion model and got a 19% validation score improvement from 37 experiments on a 0.8B parameter model, reporting results the day after he started.



The staircase chart is real, but each step is small. The agent finds genuine improvements, tweaking a beta parameter here, adding regularization there. Nobody has reported it inventing a novel attention mechanism or proposing an architectural idea that a human researcher wouldn't eventually get to.

The creativity ceiling

[GitHub Issue #22](#) captures the structural problem. A user observed that the agent cycles through minor variations of whatever worked last, stuck in a local search pattern.

The ratchet only accepts changes that immediately improve `val_bpb`, so the agent can never take a step backward to set up a larger gain.

Human researchers routinely reason, "it'll get worse before it gets better." The ratchet has no room for that.

Karpathy acknowledged a related problem on [Hacker News](#): the agent feels "*cagy and scared*" on open-ended problems.

He attributes this to [RLHF](#) (Reinforcement Learning from Human Feedback) training, which rewards safe, conservative outputs over bold experimentation. The agent is capable of proposing creative changes but built to play it safe.

The fixed 5-minute training window adds another constraint.

Changes that show their value quickly get found; changes that would only prove themselves over longer runs remain invisible. And running 100 experiments against the same validation set carries an overfitting risk: some improvements may be specific to that eval rather than genuine gains. The immutability of `prepare.py`, the system's fairness guarantee, is also its blind spot.

The community is debating whether the ceiling comes from the framework or from the underlying model. Proposals include meta-prompt optimization (a second agent rewrites `program.md` based on results), diversity directives that reward novelty alongside improvement, and periodic "reset" experiments starting from an earlier checkpoint to escape local optima.

Right now, AutoResearch is a tool that automates the methodical part of ML research: running and evaluating hundreds of small experiments. It doesn't replace the creative part, formulating new research directions, which is still a human job. That division of labor is also the clearest signal for whether this tool belongs in your workflow.

When to Use AutoResearch

The three-file contract (immutable evaluator, agent-modifiable implementation, human-authored direction) transfers beyond [LLM training](#) to any domain where you can define an automatic scoring function.

Search ranking optimization, product categorization, clinical named entity recognition, fraud scoring, intent classification: these tasks share the right traits. Small models that train in minutes, clear scoring functions, and improvements that transfer when you scale up.

When experiments run 100x faster than a human can manage, though, the eval pipeline

becomes the constraint. Static benchmarks saturate quickly. Teams adopting this pattern need eval sets that evolve alongside production data and harder edge cases.

Karpathy himself runs a "bigger cousin" of AutoResearch on 8x H100 GPUs with his production nanochat framework, which suggests the pattern scales beyond toy experiments. The community has already forked it for [macOS/Apple Silicon](#) and proposed integrations for older GPUs. The [CPU vs GPU guide](#) covers why GPU hardware is non-negotiable for this kind of iterative training.

If your goal is to squeeze incremental improvements out of a well-understood training pipeline, AutoResearch is a good fit. The creativity ceiling means you'll still need human researchers for the problems that require genuine novelty, and the tool works best on the majority of research that's methodical iteration.

Conclusion

Writing a good `program.md` requires having done the research yourself. You need to know which directions are worth trying, what "better" means for your problem, and when incremental gains have run their course. The agent handles execution, but the judgment behind the research agenda remains human. If the next generation of engineers skips that formative work because agents handle it now, the field will have plenty of compute and no one with the experience to point it in the right direction.

AutoResearch FAQs

What is AutoResearch?

AutoResearch is an open-source Python tool by Andrej Karpathy that lets an AI coding agent run ML experiments on a single GPU without human intervention. It loops through propose-train-evaluate cycles, keeping only changes that improve validation loss, and discarding everything else via git revert.

How does the AutoResearch ratchet loop work?

What is the three-file architecture in AutoResearch?



What are AutoResearch's limitations?



How does AutoResearch compare to AutoML and AlphaEvolve?



AUTHOR

Bex Tychiev

in

I am a data science content creator with over 2 years of experience and one of the largest followings on Medium. I like to write detailed articles on AI and ML with a bit of a sarcastic style because you've got to do something to make them a bit less dull. I have produced over 130 articles and a DataCamp course to boot, with another one in the making. My content has been seen by over 5 million pairs of eyes, 20k of whom became followers on both Medium and LinkedIn.

TOPICS

Artificial Intelligence AI Agents



Top DataCamp Courses

 COURSE

Designing Agentic Systems with LangChain

🕒 3 hr 👥 13.7K

Get to grips with the foundational components of LangChain agents and build custom chat agents.

[See Details →](#)

[Start Course](#)

[See More →](#)

Related

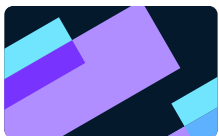
BLOG

OpenAI's Deep Research: A Guide With Practical Examples



TUTORIAL

OpenAI AgentKit Tutorial With Demo Project: Build an AI Agent



TUTORIAL

Automated Machine Learning with Auto-Keras

[See More →](#)

COMPANY

[About](#)

LEARN

[Career Tracks](#)

FOR BUSINESS

[Data & AI Skills](#)

PLANS

[Pricing](#)

TECHNOLOGIES

[Python](#)



GET THE MOBILE APP



[Privacy Policy](#) [Cookie Notice](#)

Do Not Sell My Personal Information Accessibility Security

Terms of Use